



COSC 520 Assignment 1

The Login Checker Problem

The objective of the assignment is to: (1) review hash tables, (2) explore advanced data structures, and (3) numerically compare the algorithms.

The login checker problem is to quickly check that a login name has not been already taken so that all logins are unique. Assume you have millions of logins (aka usernames), and a user creates a new account. You need to ensure the new login is not a duplicate of an existing one. Using your existing knowledge of algorithms and data structures you can

1. store all logins in a list, and check existence using linear search;
2. store all logins in a sorted array, and check existence using binary search;
3. store logins in a hash table, and check existence by hashing.

The assignment explores 2 more advanced data structures that scale better than the above. Assume the number of logins is $n=1$ billion (you may use a smaller value of n depending on your computer memory and speed as long as you justify the value of n used to validate your approach).

- Read about [bloom filters](#) and look at [examples](#).
- Read about [Cuckoo filters](#) and how they [compare](#) to Bloom filters.

Submission

To complete the assignment, submit the following. Note that you are expected to implement all five data structures yourself, not using the built-in libraries.

1. A concise pdf file listing the time and space computational complexity of: linear search, binary search, hashing, Bloom filter, and Cuckoo filter. Use appropriate parameters to describe the complexity, e.g., Bloom filter uses n (estimated number of elements/logins), m (bit array of m bits), and k (number of hash functions). You need to justify the complexity; explain each parameters used and provide the formula with appropriate references.



a place of mind
THE UNIVERSITY OF BRITISH COLUMBIA

2. Plots showing the run time complexity for large enough data. Aim for 1 billion (you may use a smaller value of n depending on your computer memory and speed as long as you justify the value of n used to validate your approach). You can generate strings dataset using synthesized functions. Then, run your program with different implementations using linear search, binary search, hashing, Bloom filter, Cuckoo filter, and compare their run time in a plot. Include this analysis in your pdf file and explain if it supports your analysis. Remember to upload your dataset on the web and include a link to the dataset in the pdf file.
3. Python code comparing the hashing, Bloom filter, and Cuckoo filter. Add the link of your GitHub repository for this assignment to the pdf file. Your code should
 - Be well documented and clean.
 - Include unit tests.
 - follow common practises:
 - appropriate class/variable/method names (not too long and meaningful),
 - appropriate comments
 - comments for each method should indicate input, output, and a short explanation of the method
 - clear setup and running instructions.

How to submit: Submit your PDF file to Canvas, which should also include the links to your GH repository and the dataset you used for testing. Use the [Association for Computing Machinery \(ACM\) - Small Standard Format Template](#) (Overleaf) for your report. Remove the ACM, journal, and copyright information. The report should be between 5-10 pages, including plots and references.

Usage of GenAI: The use of GenAI is discouraged. However, if you still decide to use online sources or GenAI to generate your code, first, make sure it is correct and executable. Second, make sure to include the resources you have used and/or mention that you used GenAI in your report. There is no penalty of using available code, the only drawback can be that you might not be engaged as writing the code yourself for your future references. If you still want to use GenAI, use it as an assistant. Only prompting GenAI to generate all the algorithms and test cases is not allowed. You should show which part of the code is yours and which part is generated by AI.

Usage of existing libraries: You are not allowed to use the existing libraries that implement these algorithms/data structures (so avoid just a quick function call!).



a place of mind
 THE UNIVERSITY OF BRITISH COLUMBIA

Grading rubric

	Weights	Subtotals
Report		20
Complexity analysis	10	
Comparisons	8	
References	2	
Code static		35
naming	5	
comments	10	
running instructions	5	
code design	15	
Code execution		35
syntax-error free, runs	10	
unit tests run	5	
convincing demo	5	
performance	5	
comparison of the data structures (item 2 in the above list)	10	



a place of mind
THE UNIVERSITY OF BRITISH COLUMBIA

Cross-Check **		10
Examining and running one of your classmate's assignment and grade it based on the above criteria. It will be assigned to you by the instructor.	10	
Total	100	100

**Cross-Check needs to be submitted within one week of the assignment submission deadline. Other than just checking, use it as a learning opportunity. Include a rationale and degradation feedback and submit it through email to the instructor: fatemeh.fard@ubc.ca.

Bonus

Extra points are 15% of your assignment mark and are added to the assignment grade.

1. Suggest an alternative competitive method; explain its data structure, how it works, and its complexity with appropriate reference. Note that receiving the bonus mark depends on the approval of instructor [15% bonus].
2. Discuss your alternative method with the instructor, and if the instructor considers your method competitive, implement it and benchmark it vs. Bloom and Cuckoo [15% bonus].

Submission deadline:

Friday Oct 2nd, 2026, at 11:59 PM.